◆ Assignment 04

Assignment 4 — File I/O, Serialization, EF & LINQ

Modules 6–7 · 10 points

 Deadline: Week 5 — Thursday, 11:59 PM

Scenario

You continue building the **Grades Prototype**. Today you persist data to files and query it with **LINQ**. This is direct preparation for **EF Core and Web API** in the upcoming courses.

Recall the `SELECT ... WHERE ...` queries from the SQL course — you'll write the same logic here, but from inside C#.

This assignment covers: **File I/O**, **JSON serialization**, and **EF** / **LINQ** concepts.

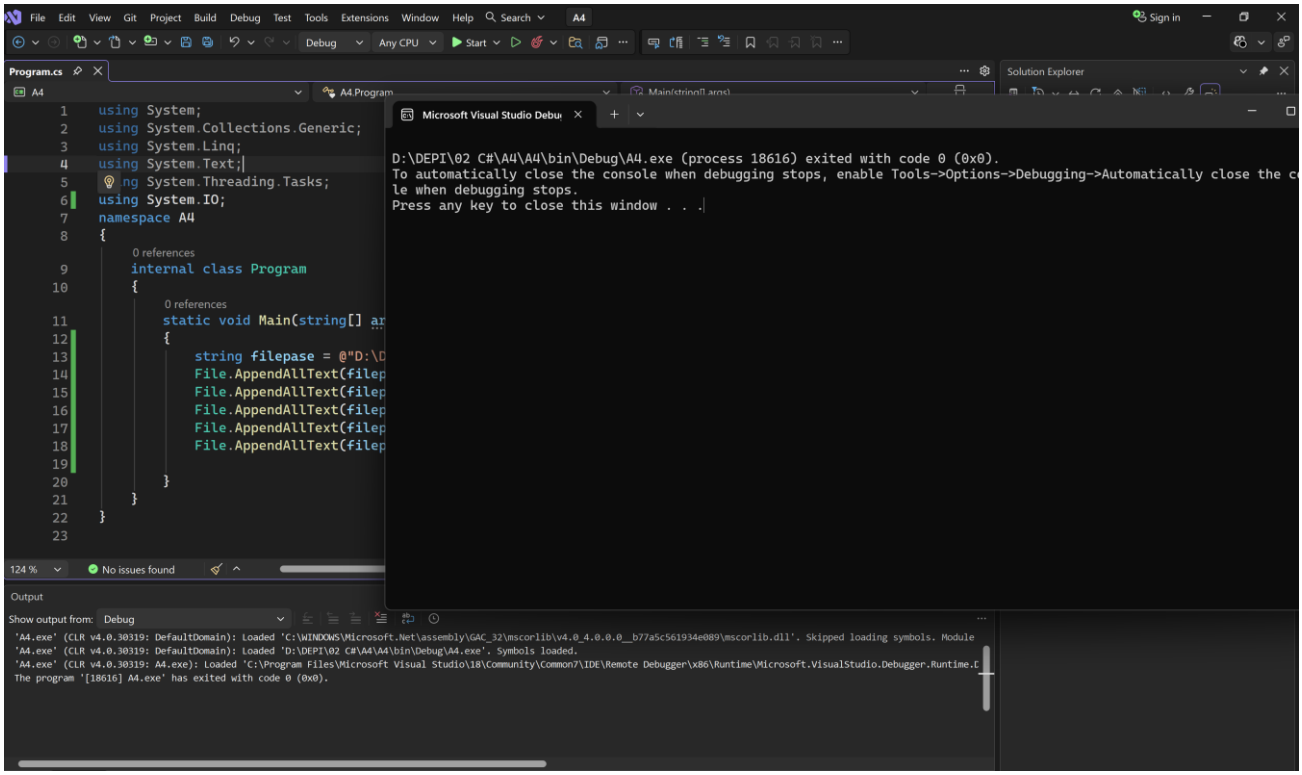
Part 1 — File I/O (3 points)

1. Write 5 student names to `students.txt` using `File.WriteAllLines`.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.IO;
namespace A4
{
   internal class Program
   {
      static void Main(string[] args)
      {
         string filepase = @"D:\DEPI\02 C#\A4\student.txt";

         File.AppendAllText(filepase, "Mohamed\n");
         File.AppendAllText(filepase, "Ali\n");
         File.AppendAllText(filepase, "Yossef\n");
         File.AppendAllText(filepase, "Ahmed\n");
         File.AppendAllText(filepase, "Reda\n");

      }
   }
}
```



2. Read the file with `File.ReadAllLines` and print each name with its number using `for`.

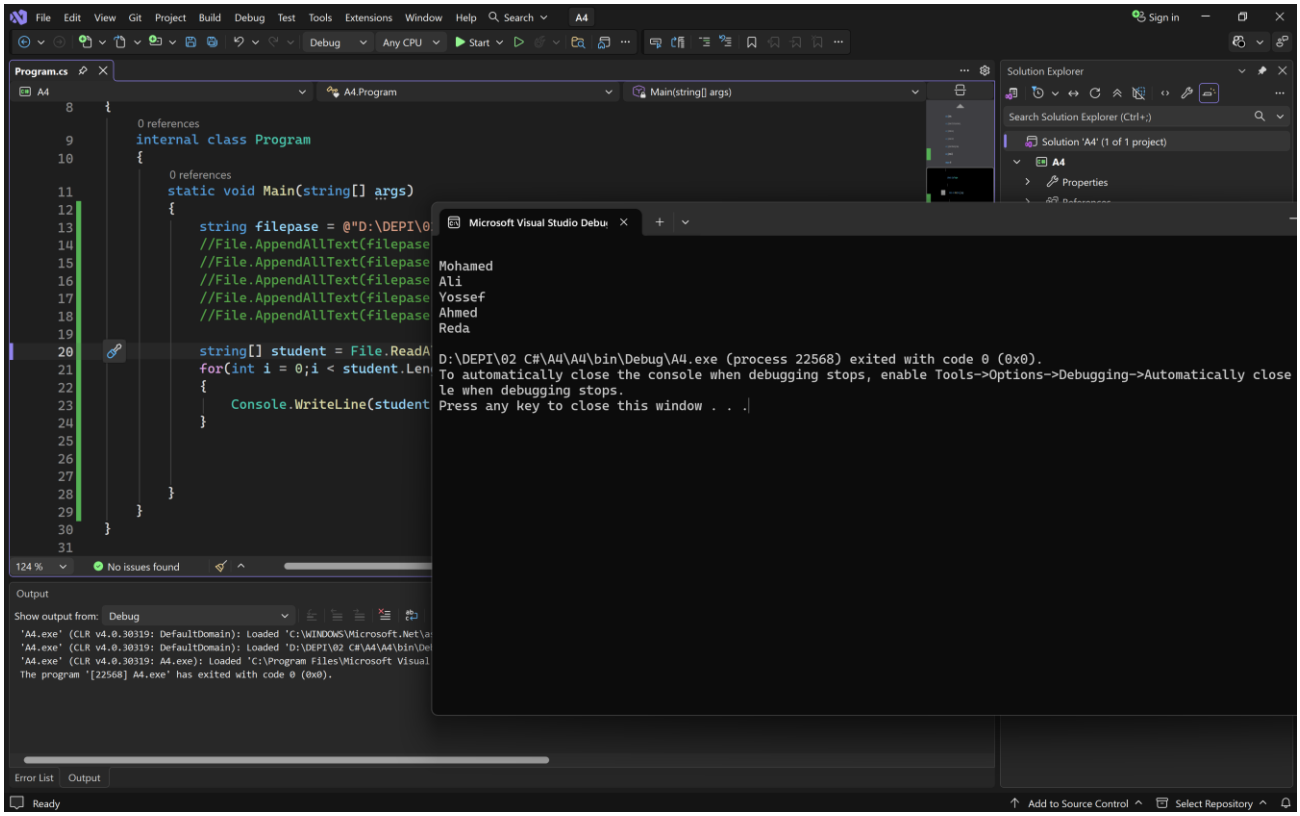
```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.IO;
namespace A4
{
   internal class Program
   {
      static void Main(string[] args)
      {
         string filepase = @"D:\DEPI\02 C#\A4\student.txt";

         File.AppendAllText(filepase, "Mohamed\n");
         File.AppendAllText(filepase, "Ali\n");
         File.AppendAllText(filepase, "Yossef\n");
         File.AppendAllText(filepase, "Ahmed\n");
         File.AppendAllText(filepase, "Reda\n");


         string[] student = File.ReadAllLines(filepase);
         for(int i = 0;i < student.Length; i++)
         {
            Console.WriteLine(student[i]);
         }



      }
   }
}
```

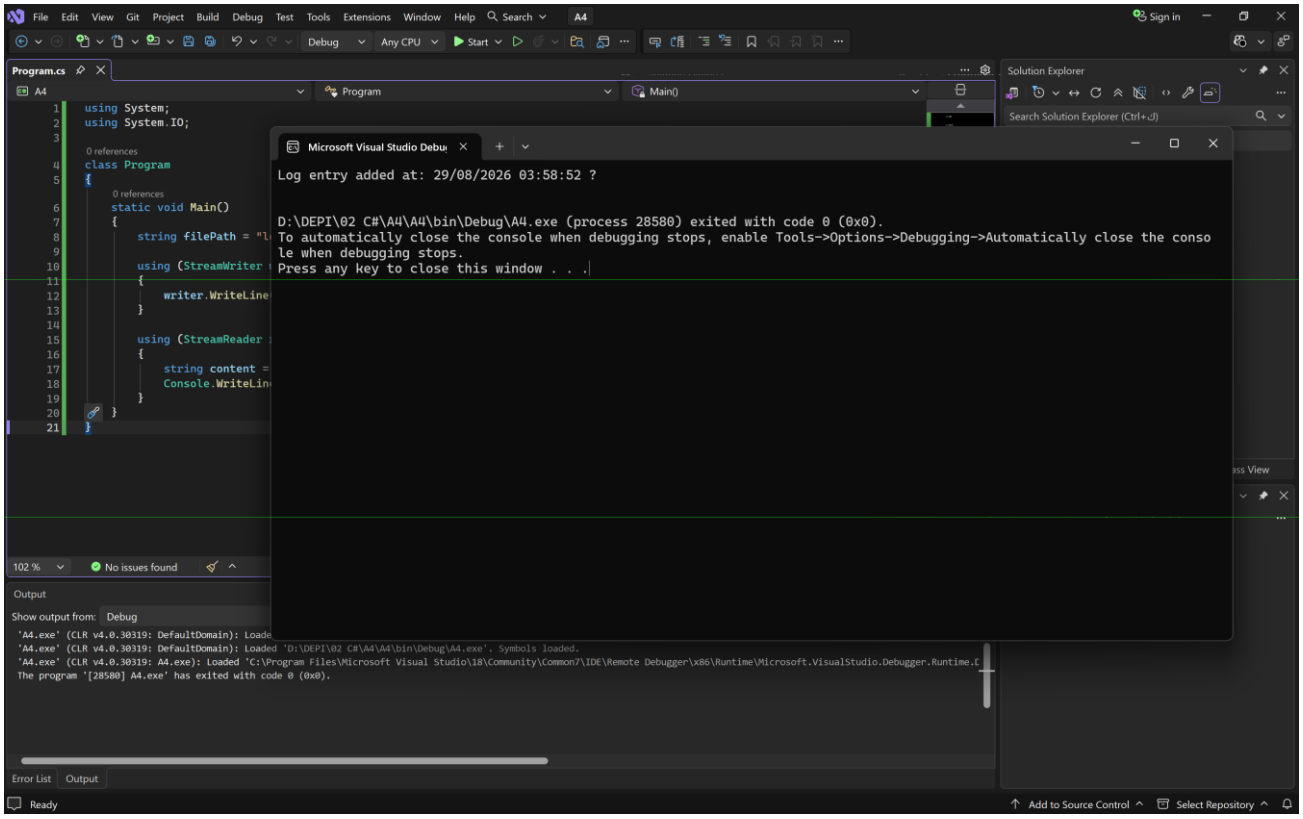


3. Use `StreamWriter` (inside `using`) to append a new line to `log.txt`, then read it with `StreamReader`.

```
using System;
using System.IO;
class Program
{
   static void Main()
   {
      string filePath = "log.txt";

      using (StreamWriter writer = new StreamWriter(filePath, append: true))
      {
         writer.WriteLine($"Log entry added at: {DateTime.Now}");
      }

      using (StreamReader reader = new StreamReader(filePath))
      {
         string content = reader.ReadToEnd();
         Console.WriteLine(content);
      }
   }
}
```



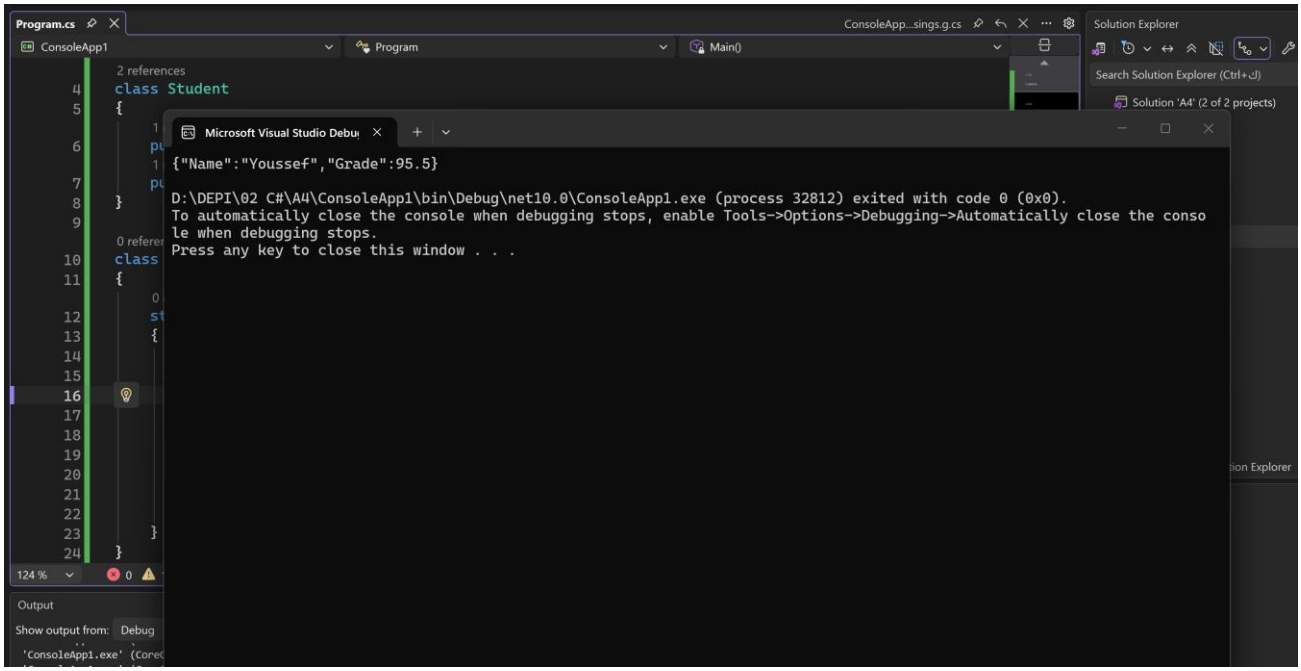
Part 2 — JSON Serialization (2 points)

4. Create a class `Student` with `Name` and `Grade`. Serialize an object of it to **JSON** with `JsonSerializer.Serialize` and print the result.

```
using System;
using System.Text.Json;
class Student
{
   public string Name { get; set; }
   public double Grade { get; set; }
}
class Program
{
   static void Main()
   {
      Student student = new Student
      {
         Name = "Youssef",
         Grade = 95.5
      };

      string jsonString = JsonSerializer.Serialize(student);

      Console.WriteLine(jsonString);
   }
}
```

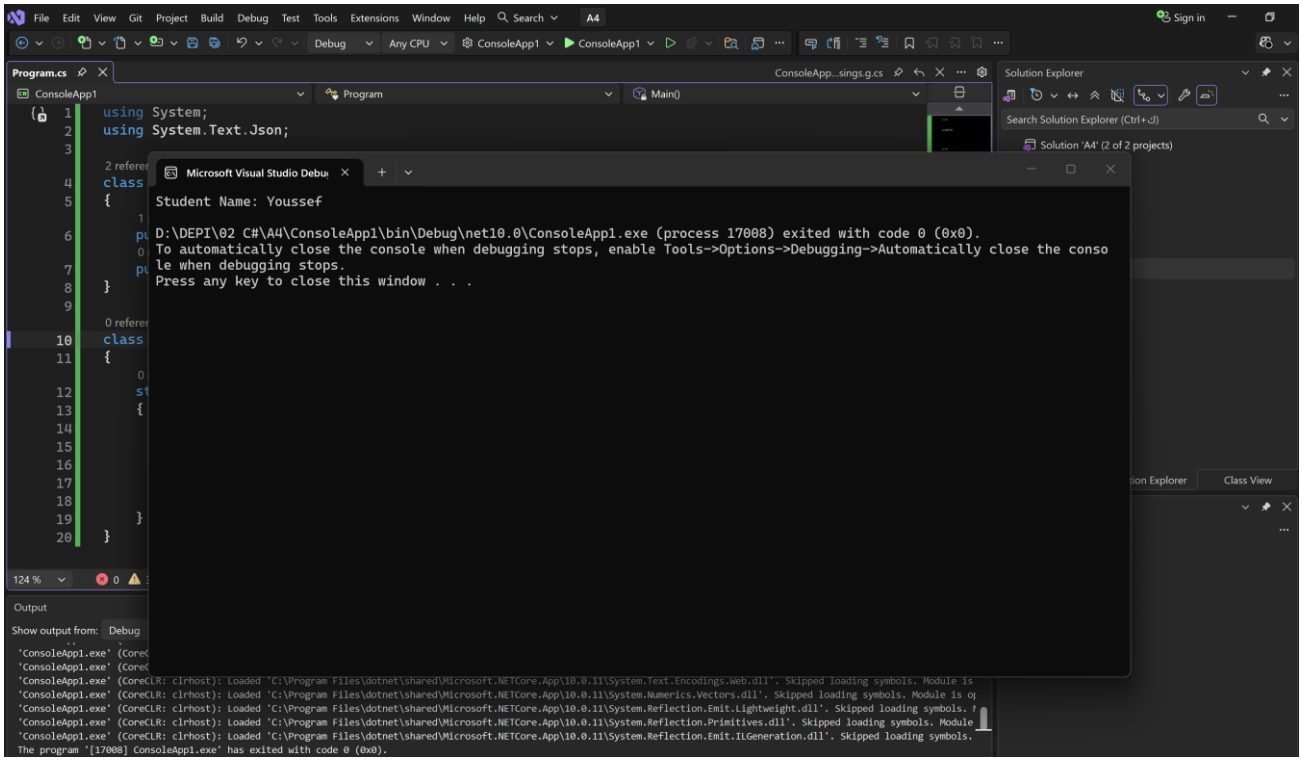


5. Deserialize the JSON back into an object with `Deserialize` and print the `Name` to confirm it round-tripped correctly.

```
using System;
using System.Text.Json;
class Student
{
   public string Name { get; set; }
   public double Grade { get; set; }
}
class Program
{
   static void Main()
   {
      string jsonString = "{\"Name\":\"Youssef\",\"Grade\":95.5}";

      Student student = JsonSerializer.Deserialize<Student>(jsonString);

      Console.WriteLine($"Student Name: {student.Name}");
   }
}
```



Part 3 — LINQ (3 points)

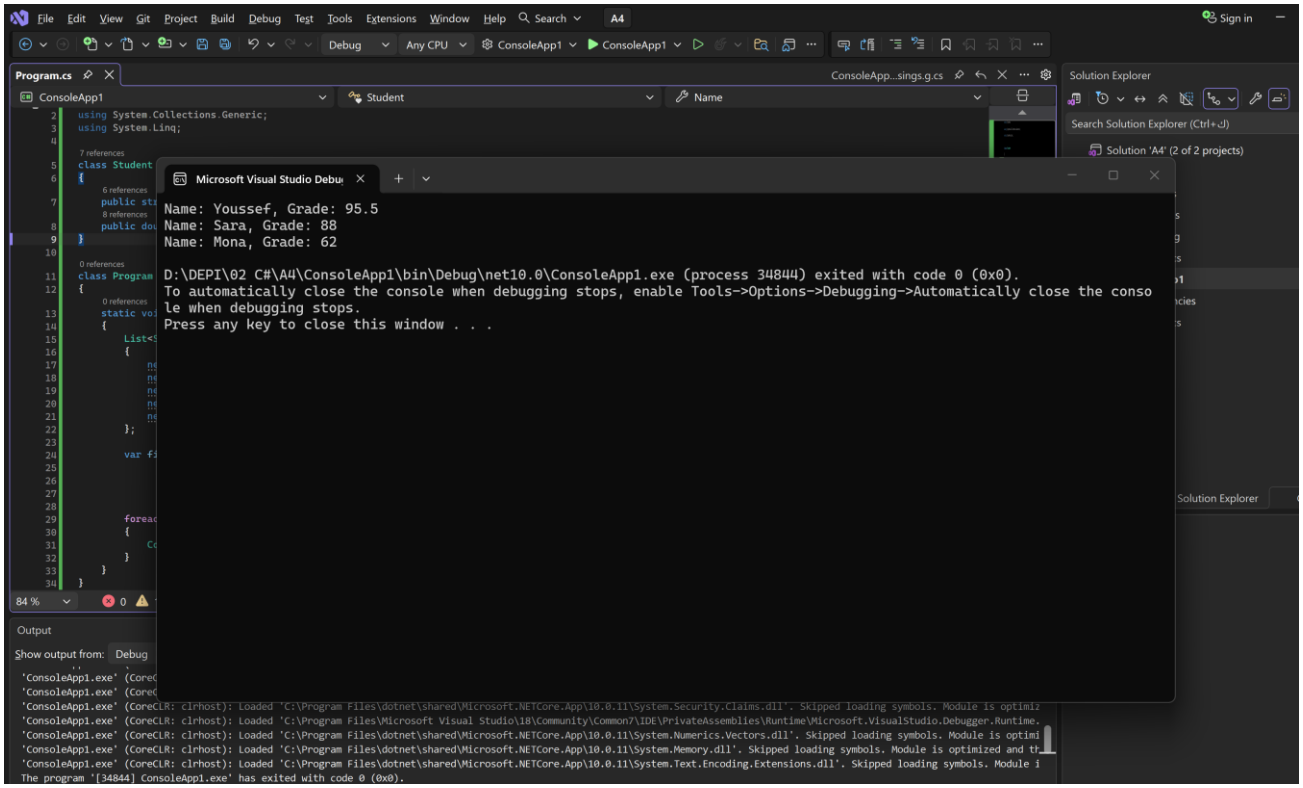
Create a `List<Student>` with 5 students of varying grades, then apply:

6. A query returning students with grade `>` `50` ordered descending — use **query syntax** (`from...where...orderby...select`).

```
using System;using System.Collections.Generic;using System.Linq;
class Student
{
   public string Name { get; set; }
   public double Grade { get; set; }
}
class Program
{
   static void Main()
   {
      List<Student> students = new List<Student>
      {
         new Student { Name = "Ahmed", Grade = 45.0 },
         new Student { Name = "Youssef", Grade = 95.5 },
         new Student { Name = "Mona", Grade = 62.0 },
         new Student { Name = "Khaled", Grade = 30.0 },
         new Student { Name = "Sara", Grade = 88.0 }
      };

      var filteredStudents = from s in students
                             where s.Grade > 50
                             orderby s.Grade descending
                             select s;

      foreach (var student in filteredStudents)
      {
         Console.WriteLine($"Name: {student.Name}, Grade: {student.Grade}");
      }
   }
}
```

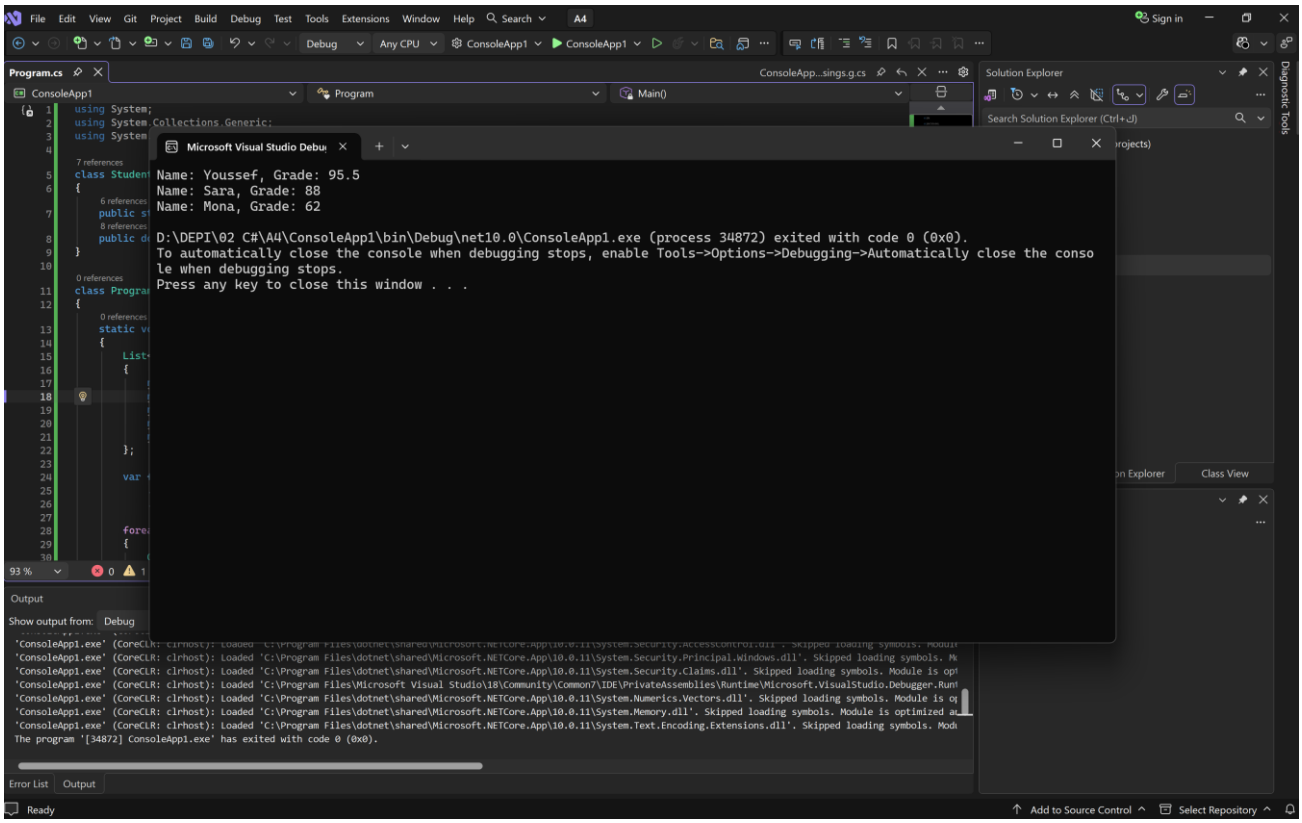


7. The same query using **method syntax** (`.Where(...).OrderByDescending(...)`).

```
using System;
using System.Collections.Generic;
using System.Linq;
class Student
{
   public string Name { get; set; }
   public double Grade { get; set; }
}
class Program
{
   static void Main()
   {
      List<Student> students = new List<Student>
      {
         new Student { Name = "Ahmed", Grade = 45.0 },
         new Student { Name = "Youssef", Grade = 95.5 },
         new Student { Name = "Mona", Grade = 62.0 },
         new Student { Name = "Khaled", Grade = 30.0 },
         new Student { Name = "Sara", Grade = 88.0 }
      };

      var filteredStudents = students
         .Where(s => s.Grade > 50)
         .OrderByDescending(s => s.Grade);

      foreach (var student in filteredStudents)
      {
         Console.WriteLine($"Name: {student.Name}, Grade: {student.Grade}");
      }
   }
}
```

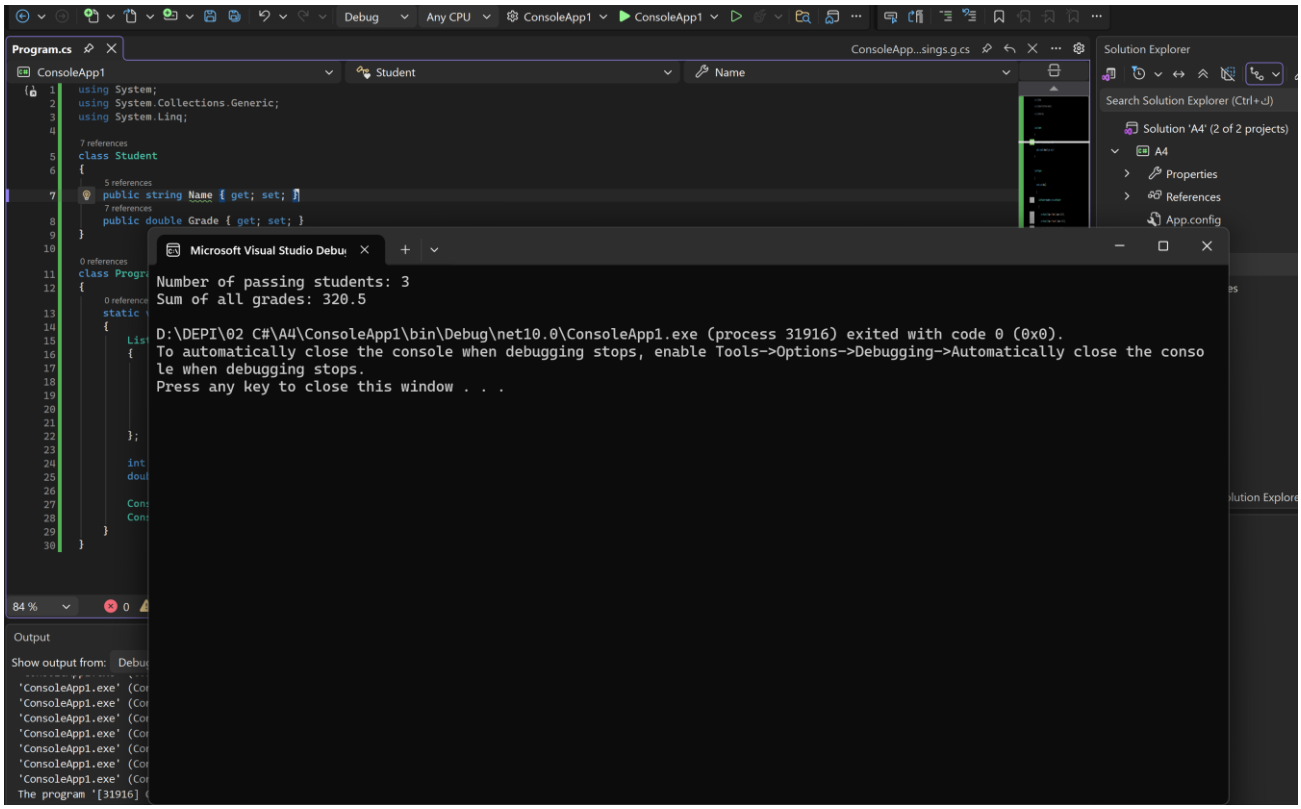


8. Compute `.Count()` of passing students and `.Sum(...)` of all grades.

```
using System;
using System.Collections.Generic;
using System.Linq;
class Student
{
   public string Name { get; set; }
   public double Grade { get; set; }
}
class Program
{
   static void Main()
   {
      List<Student> students = new List<Student>
      {
         new Student { Name = "Ahmed", Grade = 45.0 },
         new Student { Name = "Youssef", Grade = 95.5 },
         new Student { Name = "Mona", Grade = 62.0 },
         new Student { Name = "Khaled", Grade = 30.0 },
         new Student { Name = "Sara", Grade = 88.0 }
      };

      int passingCount = students.Count(s => s.Grade >= 50);
      double totalGradesSum = students.Sum(s => s.Grade);

      Console.WriteLine($"Number of passing students: {passingCount}");
      Console.WriteLine($"Sum of all grades: {totalGradesSum}");
   }
}
```



Part 4 — Comprehension (2 points)

9. What does **Entity Framework** do? What is the difference between `database-first` and `code-first`?

Entity Framework (EF) is an ORM framework in .NET that lets developers work with databases using C# objects and LINQ instead of writing raw SQL queries.

Database-First vs. Code-First:

-

Database-First: You design the database on SQL Server first, then EF generates the C# classes for you.

-
-

Code-First: You write the C# classes first, then EF creates and updates the database using Migrations.

-

10. Compare LINQ query syntax to SQL: which LINQ keyword maps to `SELECT` and which to `WHERE`?

select , where

 Submission: .cs file + screenshots + the `students.txt` file, posted to the group.

 Tip: **Working** code that you understand line by line is what matters — not perfect code. Use `breakpoints` if something doesn't run.